

Project One - Stats and Levels!



Context:

Games are big! So we can't start off making World of Warcraft, instead consider that a game is made up of multiple systems, i.e. combat, shop, etc. As we are starting out in our coding journey we can start by emulating systems, with the goal that we can piece multiple systems together to make a bigger game.

This assignment is designed to prove that you can manipulate variables, use functions and look after a very simple script without turning it into an incomprehensible mess. It'll also test your ability to code creatively AND work directly to spec.

You are to create and emulate a battle system that incorporates stats and level management systems for a simple turn based RPG; and displaying this information to the player on screen.

You're only just learning how to code, so this game won't be very complex or have super exciting graphics- however we can hopefully use it to prove some code basics and have some fun!

Artificial Intelligence Use:

- **Level 1 - No AI:** Students do not use AI tools in their creative process.

How to Guides:

- [How to add flowchart into documentation.](#)
- [How to do a recording using Zoom.](#)
- [How to screenshare in Zoom](#)
- [How to create a pdf of documentation.](#)
- [How to zip up your unity project.](#)
- [How to create a zip file and submit your project.](#)

Minimum Requirements:

Tick off the following requirements to make sure your Project is complete.

[Project Documentation](#) Requirements:

- ☐ Flow Chart(s) that show step-wise logic and branching decisions with outcomes.
- ☐ Pseudo code planning of your scripts and provides guidance on the overall steps that your program will perform.
- ☐ Inclusion of your APA7 Reference.

Post Project Reflective Requirements:

- ☐ A MP4 video covering and discussing and reflecting on the following:
 - ☐ Showing the Authoritative source that you have reviewed and what you learned from it and how you could apply it to your current project.
 - ☐ Showing your APA 7 Reference on screen.
 - ☐ Show and critically analysing your work and making suggestions for **actionable steps** for improvement:
 - ☐ Pseudo Code.
 - ☐ Flow Chart
 - ☐ Code and Unity Project.
 - ☐ Reviewing your code and showing where you have completed each of the program requirements in your submission.
 - ☐ Show the program running in Unity Editor, even if errors occur.
 - ☐ An estimation of the grade you believe you should receive based on your submission.
 - ☐ 3-7 mins max Length.

Program Requirements:

- ☐ Player (Your Bugémon: Bool-basaur)
 - ☐ Core Stats:
 - ☐ attackStrength: must be an int at the start (can become a float after leveling).
 - ☐ level: starts at 1.
 - ☐ EXP: increases when enemies are defeated.
 - ☐ EXP threshold: starts small and increases each time you level up.
 - ☐ Attacking:
 - ☐ When the player presses a key (e.g. [A] for Attack), Bool-basaur deals damage equal to their attack stat to the current enemy's HP.
 - ☐ Leveling Up:

- ☐ When Bool-basaur's EXP reaches or exceeds the threshold, a prompt should appear:
 - ☐ "Level-up ready! Press [Space] to evolve Bool-basaur to the next level..."
- ☐ When the player presses [Space], the following should happen:
 - ☐ Increase Bool-basaur's level by 1.
 - ☐ Multiply the current attack stat by 1.25 (i.e., increase it by 25%).
 - ☐ Increase the EXP threshold for the next level.
- ☐ When Bool-basaur reaches Level 5, the game ends and you should print a win message such as:
 - ☐ "Congratulations! Bool-basaur evolved into Struct-asaur!"
- ☐ Enemies (Wild Bugémon)
 - ☐ Enemies "appear" one at a time.
 - ☐ Each enemy has:
 - ☐ A random level
 - ☐ A HP value based on their level (e.g., Level 1 = 10 HP, Level 2 = 15 HP, etc.)
 - ☐ When an enemy's HP drops below 1:
 - ☐ Print a message that the enemy has fainted.
 - ☐ Grant Bool-basaur a random amount of EXP.
 - ☐ "Spawn" a new enemy immediately (until Bool-basaur reaches Level 5).
- ☐ Debug Log (Output to Console) and UI Requirements :
 - ☐ Your game must print the following to the Unity console using Debug.Log() and show it on screen using Text:
 - ☐ When EXP is gained:
 - ☐ " Bool-basaur gained 12 EXP!"
 - ☐ When a level-up is available:
 - ☐ "Level-up ready! Press [Space] to evolve Bool-basaur to the next level..
 - ☐ When leveling up:
 - ☐ "Level up! Bool-basaur is now Level 2!"
 - ☐ "Attack increased to 12.5"
 - ☐ "Next level EXP threshold: 40"
 - ☐ When damage is dealt:
 - ☐ " Bool-basaur used Logic Lash! It dealt 10 damage!"
 - ☐ "Byte-chu's HP is now 5"
 - ☐ When a new enemy appears:
 - ☐ "A wild Byte-chu appeared! Level 2 with 15 HP!"
 - ☐ When Level 5 is reached:

- ☐ " Bool-basaur evolved into Struct-asaur!
- ☐ Your system needs to utilise a sprites for:
 - ☐ Player
 - ☐ Enemy

Example Debug Log Output:

```
A wild Bytechu appeared! Level 2 with 15 HP!

Boolbasaur used Logic Lash!
It dealt 10 damage!
Bytechu's HP is now 5

Boolbasaur used Boolean Bomb!
It dealt 10 damage!
Bytechu's HP is now -5
The wild Bytechu fainted!
Boolbasaur gained 12 EXP!

Total EXP: 12 / 20

A wild Nullkit appeared! Level 1 with 10 HP!

Boolbasaur used AND Slash!
It dealt 10 damage!
Nullkit's HP is now 0
The wild Nullkit fainted!
Boolbasaur gained 15 EXP!

Total EXP: 27 / 20
Level-up ready! Press [Space] to evolve Boolbasaur to the next level...

[Space pressed]

Level up! Boolbasaur is now Level 2!
Attack increased to 12.5
Next level EXP threshold: 40
```

Repeat, till level 5.

```
Level-up ready! Press [Space] to evolve Boolbasaur to the next level...

[Space pressed]

Level up! Boolbasaur is now Level 5!
Attack increased to 24.41

Boolbasaur is evolving...

Congratulations! Boolbasaur evolved into Structasaur!

Structasaur has learned:

- Control Flow Vines: Deals extra damage to lower-level enemies.
- Compile Bloom: A powerful strike that only works when HP is full.
- Static Root: Stuns the enemy and skips their next attack.
```

Tips

- To spawn a new enemy, it can be as simple as printing "Oh no a wild Byte-chu appeared!" to the console and resetting the enemy's health.
- Don't forget to comment your code!
- Combining a level-up multiplier and a damage integer is a complex thing to do. You will need to convert both values to the same type (a float most likely) to multiply them, and then convert that result to an integer to figure out how much damage is being done.
- You can complete this entire project using just one script and that's totally fine to get everything working.
 - However, if you're feeling confident and want to follow better coding practices, you can break the system into separate scripts. This makes your code cleaner, easier to manage, and more like how real games are structured.
 - If you'd like to try this, check out the Week 4 videos on Cross Script Referencing. If you go this route, your project might include:
 - A Player script (handles player stats, level-up, EXP, etc.)
 - An Enemy script (controls HP, level, spawning)
 - Intermediate Option: A Combat script (handles attacks and damage — this can also be kept inside the Player or Enemy script if you're still learning)
- Start simple, then refactor as you go. Get it working first then clean it up later!

Stretch Goals

Please note, finish the 'minimum requirements' section then make a zip of your project in its current state to have as a backup. Below are suggestions as to what you could add to your program to add extra functionality, attempting these can contribute to higher grades as they further demonstrate your understanding and mastery of the topics. The more evidence that you can provide of your understanding the more evidence you provide to justify those grades. Please keep in mind that you add to your project it is expected that your documentation is also updated to include this.

Potential Ideas:

- Enemy Attacks Back
 - Goal: After the player attacks, the enemy retaliates.
 - Design:
 - Enemy uses a fixed or level-scaled attack stat.
 - Applies damage to the player.
 - May include a simple random element (e.g. damage range).
- Extra Stats (Toughness, Dexterity, Focus)
 - Goal: Introduce more depth to the Bugémon.
 - New Stats:
 - Toughness: Reduces incoming damage
 - Dexterity: Small chance to dodge an enemy attack
 - Focus: Boosts EXP gained
- Enemy Variety
 - Goal: Add new enemy types with different behaviors or stat patterns.
 - Examples:
 - Byte-chu: Fast attacker, low HP
 - Nullkit: Chance to dodge
 - Forloopin: Can attack twice
 - Implementation Ideas:
 - Randomly select enemy type on “spawn”
 - Use unique attack messages for each
- Multiple Attack Types
 - Goal: Add strategic options for the player.
 - Example Moves:
 - Logic Lash – standard attack
 - ShortCircuit – low damage, high accuracy if attacks have miss chance.
 - Compile Bloom – high risk/high reward (only usable at full HP)
 - Control Flow Vines – extra damage against lower-level enemies

- New Abilities on Level-Up
 - Goal: Make leveling meaningful by unlocking moves or bonuses.
 - Examples:
 - Level 3: Unlock Static Root (chance to prevent enemy's next turn)
 - Level 4: System Restore (heal a percentage of health after defeating an enemy)

Stretch Goals Tips

- Start Simple, Then Expand
 - Get the core game loop working first (attacking, enemy HP, EXP, level-ups).
 - Only move onto stretch goals once your base system is stable.
 - Add one new feature at a time, test it thoroughly, then move to the next.
- Break It Into Systems
 - Most stretch goals require you to separate your logic:
 - Player script: handles player stats, attacks, level-ups
 - Enemy script: controls enemy stats, random spawning
 - Combat script (optional): manages damage calculation and turn order
- Use Debug.Log() to Track Everything
 - Even if you add UI, continue using Debug.Log() to:
 - Track EXP, damage, and level-ups
 - Print logic behind decisions (e.g., "Dexterity dodge triggered!")
 - It helps debug unexpected behaviors

FAQ

- Q: My project at home says it is not opening in the correct version?
 - This means you've downloaded the wrong version, please make sure it is the same as what is on campus.

Step by step process for the brief

Part 1 - Planning.

- Read through this project outline document.
- [Review the documentation template](#) and understand the requirements of each section.
- Understand what is required for each deliverable and the scope of your project.
- Write down questions you have to bring to class next week.

Part 2 - Structuring.

- Create a flowchart showing how the game will run.
 - Recommend using [something like draw.io](#).
 - Add it into your documentation under Flowchart.
- [Set up your Unity Project using Source Control follow these instructions](#).
- Write some pseudo code (sentences describing how the game might run and what variables are needed).

Part 3 - Implementation

- Go over the Pseudo code and flowchart and see if there's anything that needs to change now that we know what scripts look like
- Translate any pseudo code that you know how to write into code now. This should cover
 - Defining variables (attack damage, level of player etc.)
 - Button presses you will need to capture
 - Any random ranges you may need

Part 4 - Completion

Part 4.1 - Refinement and Reflection

- Using what you learnt this week finish off your project by adding:
 - If statements
 - Basic functions
 - UI and Buttons to the screen.
- Review the Minimum Requirements section and ensure you are doing all those elements.
- **Now would be an ideal time to complete your reflective video.**
 - Make sure you cover:
 - Pseudo Code.
 - Flow Chart
 - Code
 - Includes minimum one APA 7 Authoritative reference.
 - Reviewing your code and showing where you have completed each of the program requirements in your submission.
 - Showing your project running, or not running.
 - An estimation of the grade you believe you should receive based on your submission.
 - Don't forget to discuss
 - The reference and how you could apply what you've learned to this or future projects.
 - Improvements to your planning processes (Pseudo Code/FlowChart) and your implementation (Code). Remember we want to be critically analysing our work so we can do it better next time!
 - Reviewing your code and showing where you have completed each of the program requirements in your submission.
 - Actionable steps, i.e. **what** could be improved and **how** it could be improved.
 - Suggested Resources:
 - [Pseudo Code](#)
 - [Flow Charts](#)

Part 4.2 - Submission Steps

- If you are unsure of any of these steps, review the how-to guides above.
- Create a submission folder on your desktop and include the following:
 - Project Documentation as a PDF which includes:
 - Flow chart for your project.
 - Pseudo code for your project.
 - Inclusion of your APA7 Reference.
 - An MP4 video covering and discussing and reflecting on the following:

- Showing the Authoritative source that you have reviewed and discussed what you learned from it and how you could apply it to your current project.
 - Showing your APA 7 Reference on screen.
- Critically analysing your work and making suggestions for actionable steps for improvement:
 - Pseudo Code.
 - Flow Chart
 - Code and Unity Project.
- Review your code and show where you have completed each of the program requirements in your submission.
- Show the program running in Unity Editor, even if errors occur.
- An estimation of the grade you believe you should receive based on your submission.
- 3-7 mins max Length.
 - A ZIP file of your Unity Project Files, make sure you remove the library folder before zipping.
- Create a Zip of your submission folder and upload this to the project 1 submission point on the project 1 page on Campus Online.